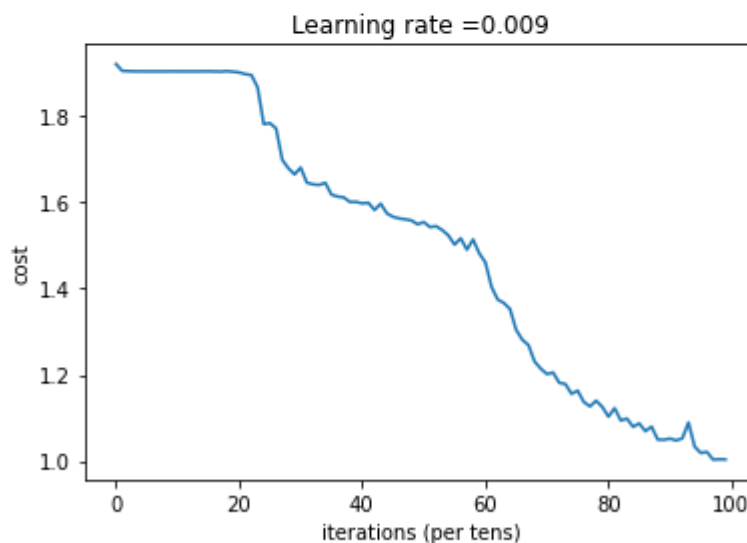


這次作業中，`forward_propagation` 所得到的值與網大的 PDF 不同，訓練出來的模型也有很大的差異，正確率不到 60%。

Adam：

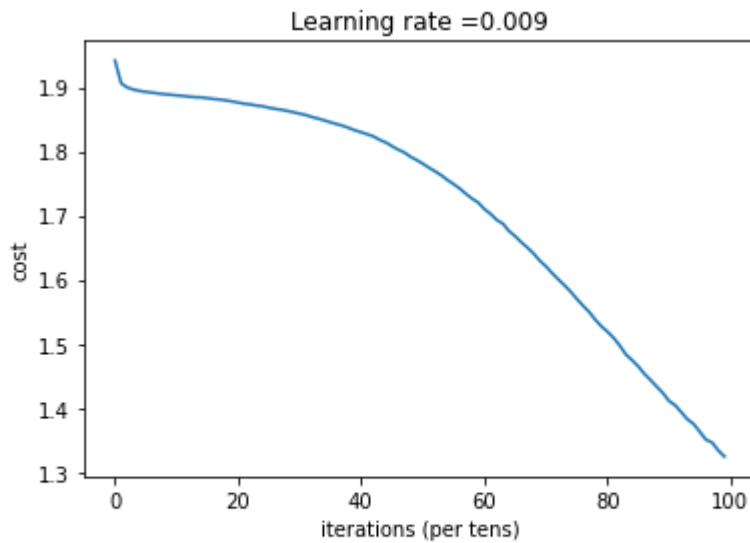
```
Cost after epoch 0: 1.921332
Cost after epoch 5: 1.904156
Cost after epoch 10: 1.904309
Cost after epoch 15: 1.904477
Cost after epoch 20: 1.901876
Cost after epoch 25: 1.784078
Cost after epoch 30: 1.681051
Cost after epoch 35: 1.618206
Cost after epoch 40: 1.597971
Cost after epoch 45: 1.566706
Cost after epoch 50: 1.554487
Cost after epoch 55: 1.502187
Cost after epoch 60: 1.461036
Cost after epoch 65: 1.304490
Cost after epoch 70: 1.201760
Cost after epoch 75: 1.163242
Cost after epoch 80: 1.102885
Cost after epoch 85: 1.087105
Cost after epoch 90: 1.051911
Cost after epoch 95: 1.018554
```



```
Tensor("Mean_1:0", shape=(), dtype=float32)
Train Accuracy: 0.6666667
Test Accuracy: 0.5833333
```

在認為其他 code 沒有問題的前提，進行替換優化器以提高成功率

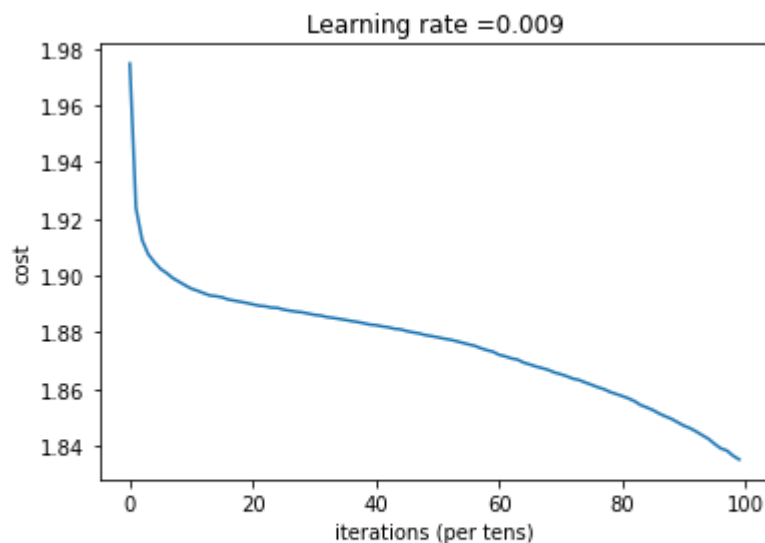
(一)adagrad



```
Tensor("Mean_1:0", shape=(), dtype=float32)  
Train Accuracy: 0.5777778  
Test Accuracy: 0.59166664
```

使用 `adagrad`，由圖可發現 `cost` 平滑下降，沒有收斂的趨勢，合理推測經過更多次的訓練能有效提高正確率。

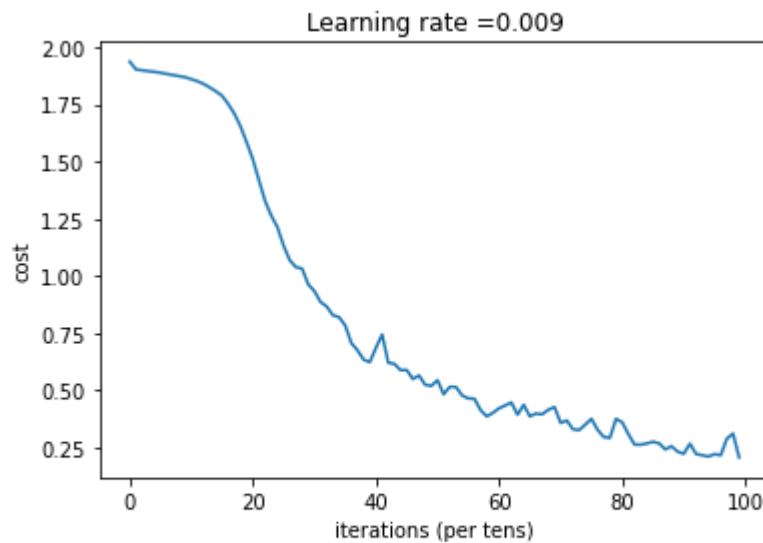
(二)gradientdescent



```
Tensor("Mean_1:0", shape=(), dtype=float32)  
Train Accuracy: 0.4101852  
Test Accuracy: 0.34166667
```

使用 `gradientdescent`，由圖可發現只有一開始下降非常快速，爾後下降緩慢，缺乏效率，且沒有收斂的趨勢，合理推測經過更多次的訓練能有效提高正確率，但考量效率不高因此不會繼續討論。

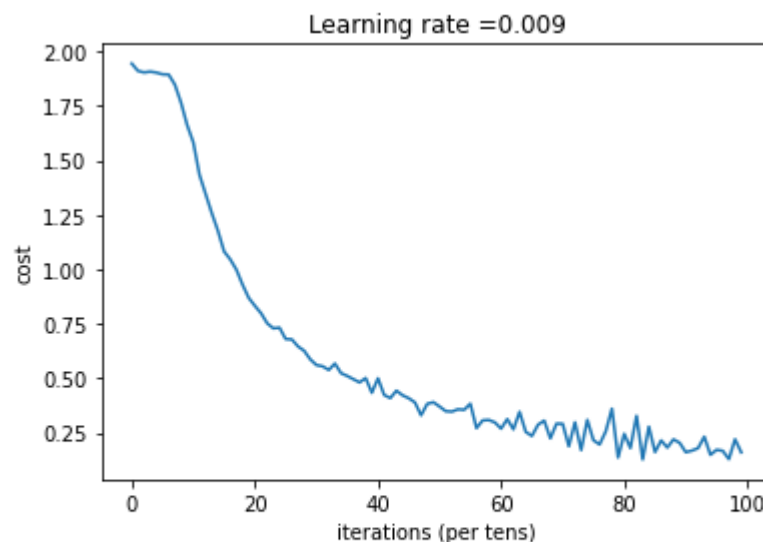
(三)momentum(momentum=0.9)



```
Tensor("Mean_1:0", shape=(), dtype=float32)  
Train Accuracy: 0.95  
Test Accuracy: 0.81666666
```

使用 momentum 可以發現，cost 下降速度非常快，正確率也是選用的優化器中最高的。

(四)RMS



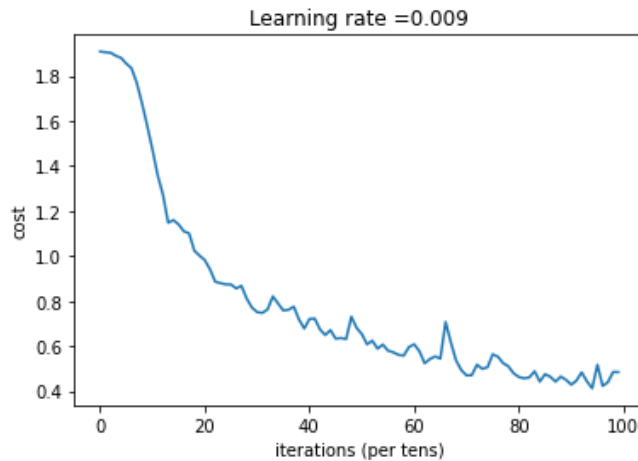
```
Tensor("Mean_1:0", shape=(), dtype=float32)  
Train Accuracy: 0.9388889  
Test Accuracy: 0.80833334
```

使用 RMSProp 的正確率略遜於 momemtun，下降速度快，後期有震盪現象。

將網路增加一層 conv2D，filter W3 的 shape=(2,2,16,8)

```
# MAXPOOL: window 4x4, stride 4, padding SAME
P2 = tf.nn.max_pool(A2, ksize=(1,4,4,1), strides=(1,4,4,1), padding="SAME")
Z3 = tf.nn.conv2d(P2, filter=W3, strides=(1,1,1,1), padding="SAME")
A3 = tf.nn.relu(Z3)
P3 = tf.nn.max_pool(A3, ksize=(1,2,2,1), strides=(1,2,2,1), padding="SAME")
```

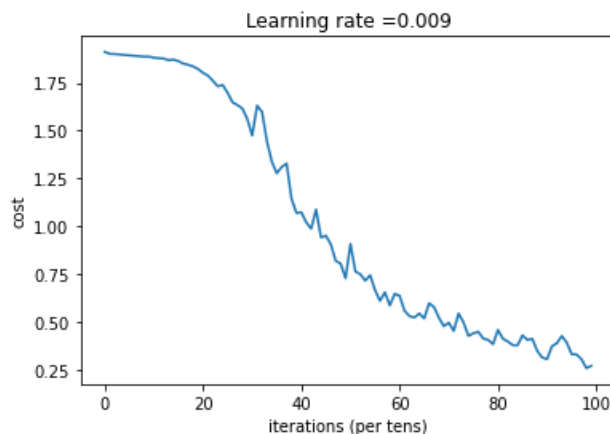
(一)adam



```
Tensor("Mean_1:0", shape=(), dtype=float32)
Train Accuracy: 0.8574074
Test Accuracy: 0.7416667
```

與原本網路的 adam 相比，正確率提高了 15%，而原本網路的 adam 後期還沒收斂，所以增加訓練次數應能繼續提高正確率。

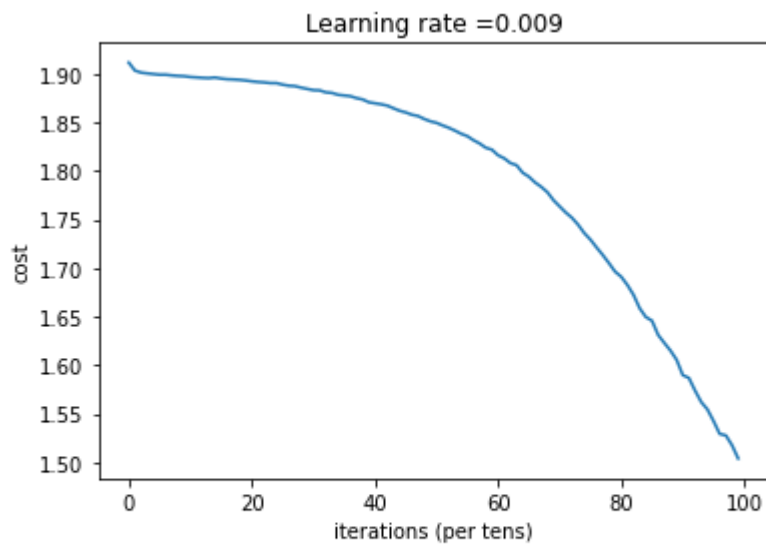
(二)momentum(momentum=0.9)



```
Tensor("Mean_1:0", shape=(), dtype=float32)
Train Accuracy: 0.88796294
Test Accuracy: 0.8
```

只有原本網路的 momentum 的正確率硬是高了一些，從這個結果可以知道增加 hidden layer 大部分的時候能提高正確率，但有些優化器並不適合。

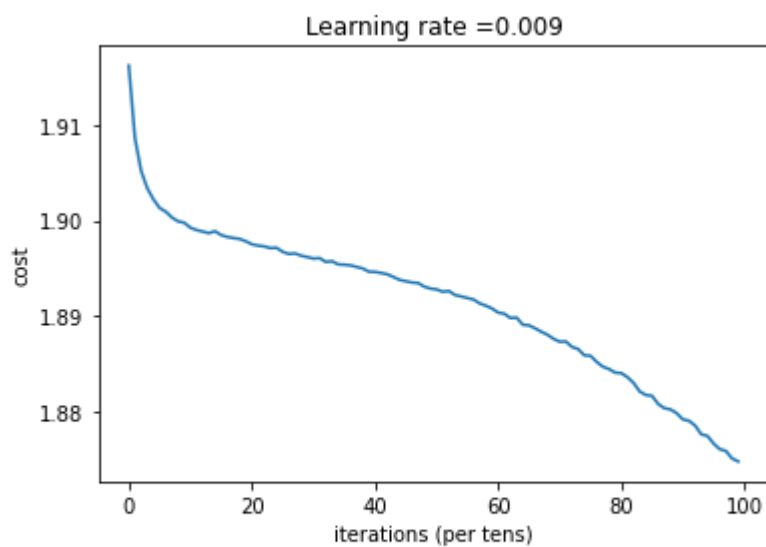
(三)Adagrad



```
Tensor("Mean_1:0", shape=(), dtype=float32)  
Train Accuracy: 0.49074075  
Test Accuracy: 0.5083333
```

跟原本網路的 `adagrad` 相比，正確率少了近 8%，`cost` 穩定下降，因此若是能增加 `epoch`，在原本網路的 `adagrad` 或多一層網路的 `adagrad` 應能提高正確率。

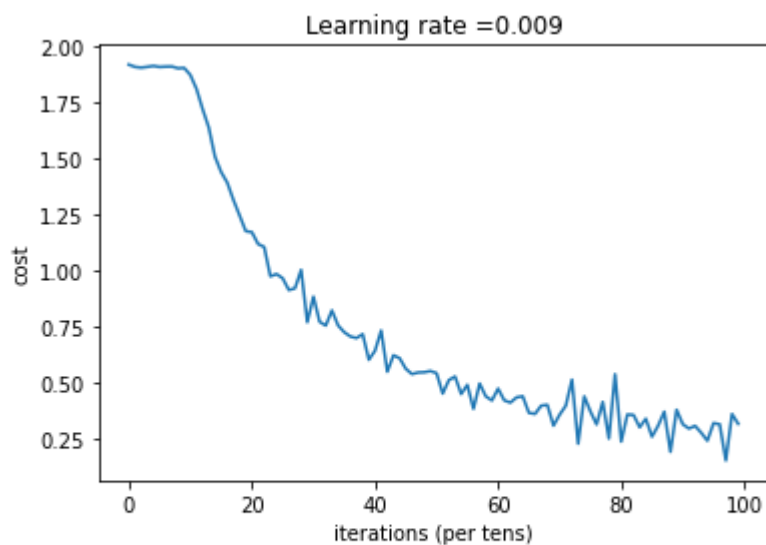
(四)`gradientdescent`



```
Tensor("Mean_1:0", shape=(), dtype=float32)  
Train Accuracy: 0.3611111  
Test Accuracy: 0.35833332
```

縱使多了一層網路，正確率依然低迷，因此不會再花額外的時間做更多次訓練。

(五)`RMSProp(decay=0.9)`



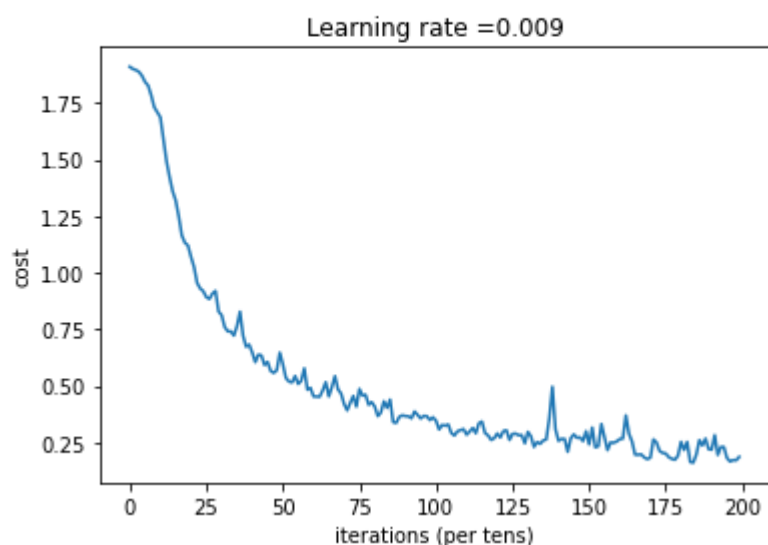
```
Tensor("Mean_1:0", shape=(), dtype=float32)  
Train Accuracy: 0.9546296  
Test Accuracy: 0.825
```

正確率是最高，也沒有 overfitting 的問題，因此接下來的測試中採用 RMSProp 應是效益最高的，後期的 cost 有震盪的出現，因此增加訓練次數應該不會有太大的差別。

將 epoch 提高到 200 次，並採用多一層 hidden layer

(一)Adam epoch=200

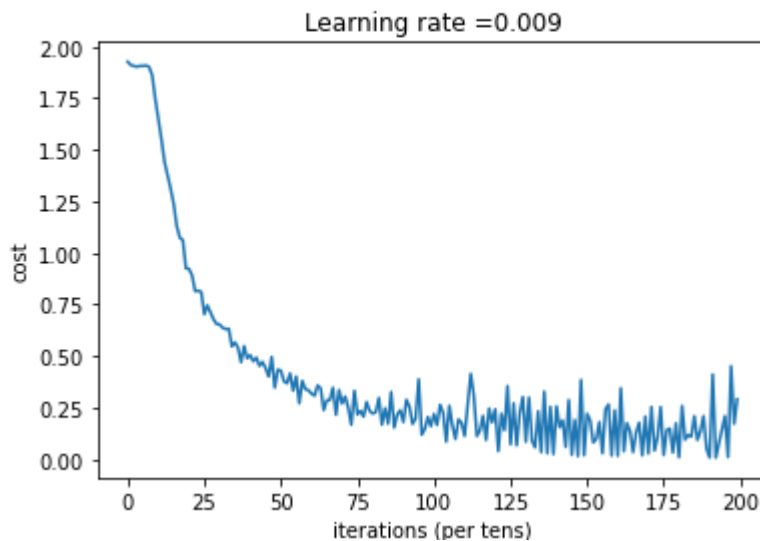
```
Train Accuracy: 0.95648146  
Test Accuracy: 0.8333333
```



效果不錯，正確率提高了 9%，因為後期開始震盪，因此不再提高訓練次數。

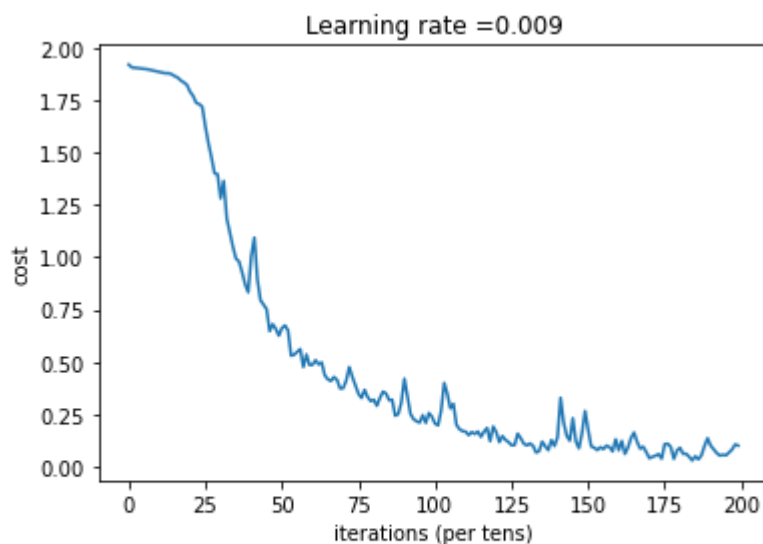
(二)RMS epoch=200

```
Train Accuracy: 0.99351853  
Test Accuracy: 0.9
```



令人意外的是，正確率達到了 90%，超過 75 次學習後震盪非常明顯，正確率應是在某一區間收斂，只是剛好 200 次學習有 90%正確率，100 次學習只有 82.5% 正確率，為了確認是否問題於此，後續測 400 次訓練時將每一次正確率顯示出來，若正確率超過 94%則停止。

(三)Momentum epoch=200



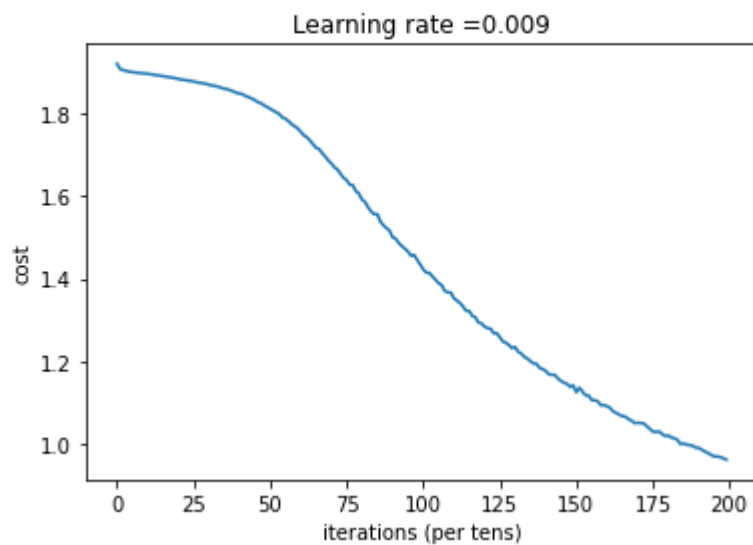
```
Tensor("Mean_1:0", shape=(), dtype=float32)  
Train Accuracy: 0.98425925  
Test Accuracy: 0.85
```

momentum 的正確率提高了 5%，訓練超過 175 次才開始收斂，震盪也愈明顯，因此增加訓練次數效益應不大。

(四)adagram epoch=200

Train Accuracy: 0.70185184

Test Accuracy: 0.65



尚未收斂，增加訓練次數能再提高正確率，效率較低。

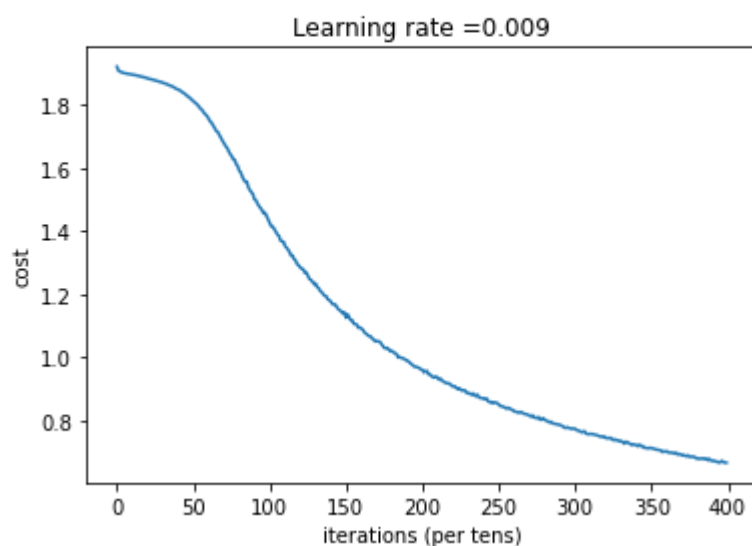
將 **epoch** 提高到 **400** 次，採用多一層 **hidden layer**，為防止因震盪使

第 **400** 次取的正確率誤差過大，設定正確率超過 **94%**則停止訓練。

(一)adagrad epoch=400

Train Accuracy: 0.8055556

Test Accuracy: 0.69166666

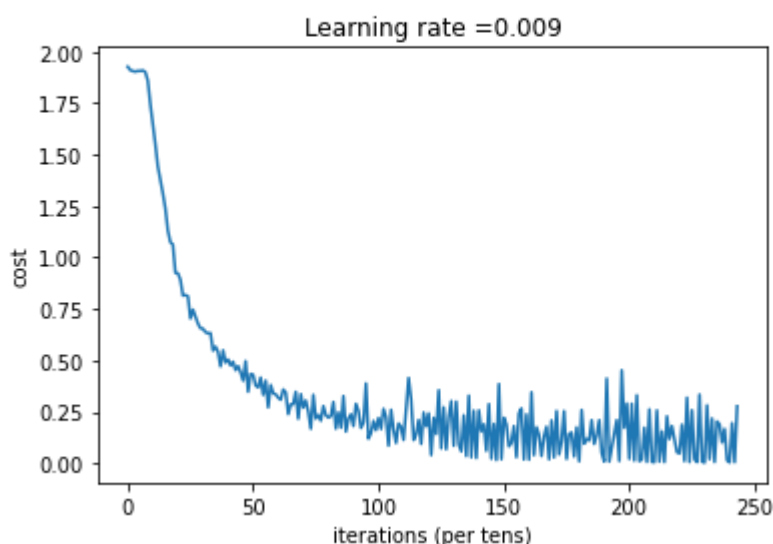


```
Cost after epoch 365: 0.694446
Train Accuracy: 0.78425926
Test Accuracy: 0.68333334
Cost after epoch 370: 0.688584
Train Accuracy: 0.80277777
Test Accuracy: 0.70833333
Cost after epoch 375: 0.684100
Train Accuracy: 0.8046296
Test Accuracy: 0.7
Cost after epoch 380: 0.679899
Train Accuracy: 0.7925926
Test Accuracy: 0.68333334
Cost after epoch 385: 0.676840
Train Accuracy: 0.8101852
Test Accuracy: 0.71666664
Cost after epoch 390: 0.671130
Train Accuracy: 0.7925926
Test Accuracy: 0.675
Cost after epoch 395: 0.667541
Train Accuracy: 0.8055556
Test Accuracy: 0.69166666
```

雖然從 cost 圖沒有收斂的趨勢，但下降速度愈來愈緩慢，再增加 200 次應會收斂，根據正確率的紀錄，epoch=300 後正確率 69%上下震盪，無法再提高。

(二)RMSProp epoch=400

```
Train Accuracy: 1.0
Test Accuracy: 0.90833336
```



(epoch=235)

```

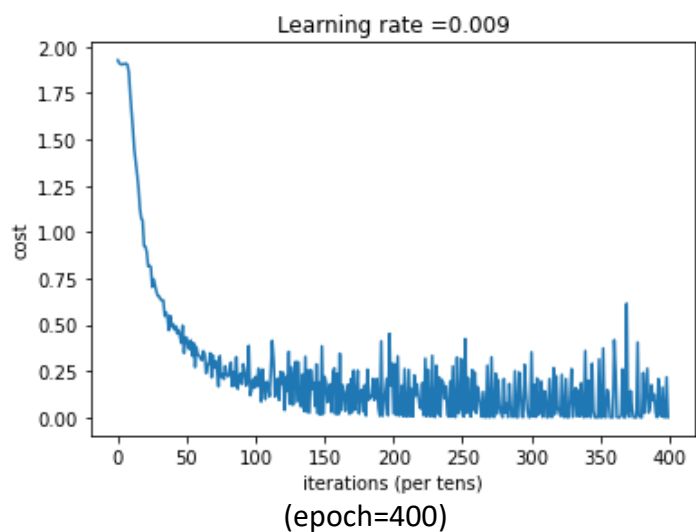
Cost after epoch 300: 0.352601
Train Accuracy: 0.9990741
Test Accuracy: 0.925
Cost after epoch 305: 0.014313
Train Accuracy: 1.0
Test Accuracy: 0.89166665
Cost after epoch 310: 0.139181
Train Accuracy: 1.0
Test Accuracy: 0.9166667
Cost after epoch 315: 0.001763
Train Accuracy: 0.9990741
Test Accuracy: 0.93333334
Cost after epoch 320: 0.035396
Train Accuracy: 0.99722224
Test Accuracy: 0.89166665
Cost after epoch 325: 0.208142
Train Accuracy: 1.0
Test Accuracy: 0.90833336
Cost after epoch 330: 0.002646
Train Accuracy: 0.9907407
Test Accuracy: 0.89166665
Cost after epoch 335: 0.005509
Train Accuracy: 0.96944445
Test Accuracy: 0.9
Cost after epoch 340: 0.002882
Train Accuracy: 0.9990741
Test Accuracy: 0.9
Cost after epoch 345: 0.036239
Train Accuracy: 1.0
Test Accuracy: 0.925
Cost after epoch 350: 0.001319
Train Accuracy: 1.0
Test Accuracy: 0.90833336
Cost after epoch 355: 0.149671
Train Accuracy: 0.75555557
Test Accuracy: 0.76666665
Cost after epoch 360: 0.418446
Train Accuracy: 1.0
Test Accuracy: 0.9166667
Cost after epoch 365: 0.013316
Train Accuracy: 0.8194444
Test Accuracy: 0.78333336

```

```

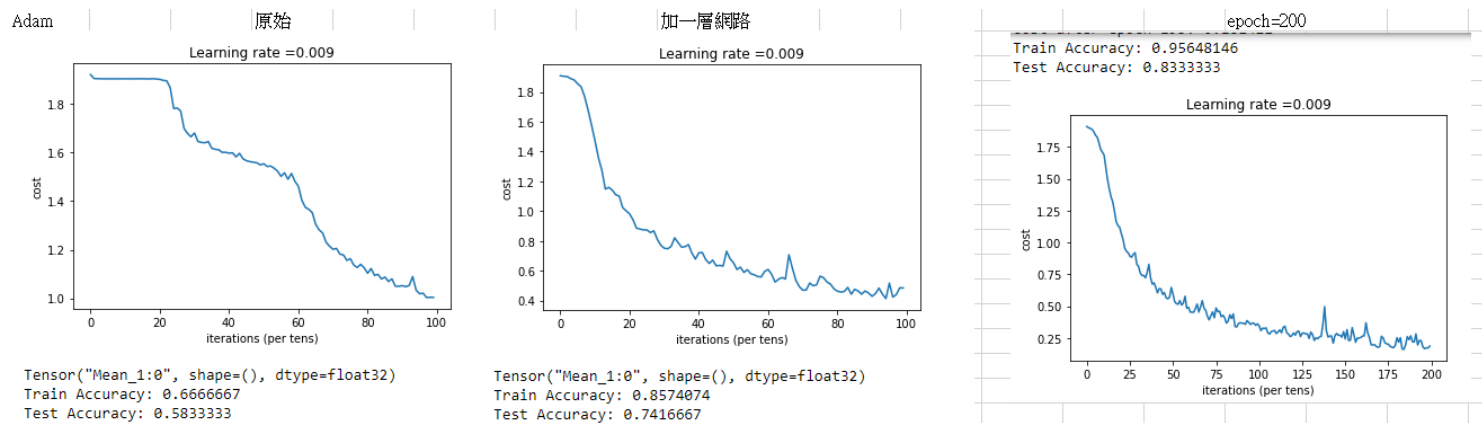
Cost after epoch 395: 0.163517
Train Accuracy: 0.9990741
Test Accuracy: 0.89166665

```

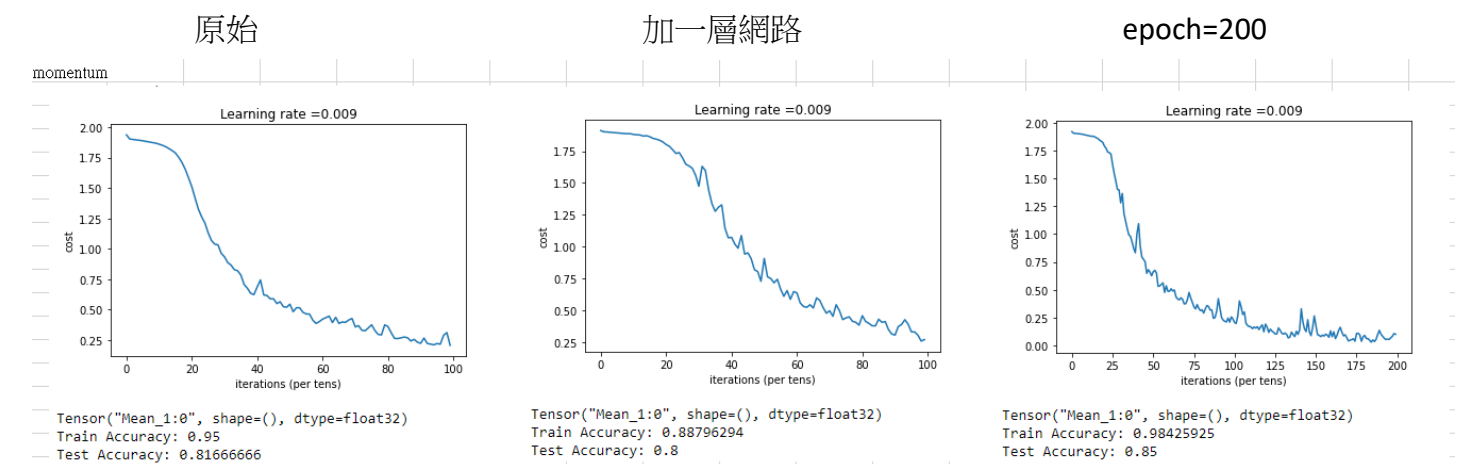


Cost 圖能看到後期震盪嚴重，調閱歷史，經過 170 次訓練後，正確率在 9 成附近震盪，無法突破 94%，完整跑完 400 次訓練，其中最高為第 315 次，來到 93%，爾後來回不定，甚至會低於 8 成(第 365 次)，因此當震盪出現時，添加限制才能取得最好的表現。

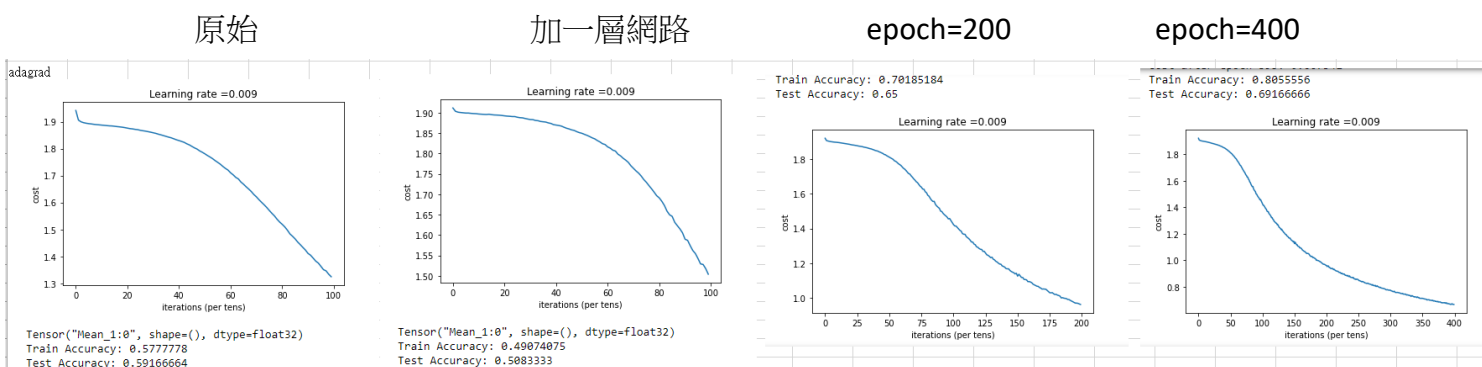
結論：



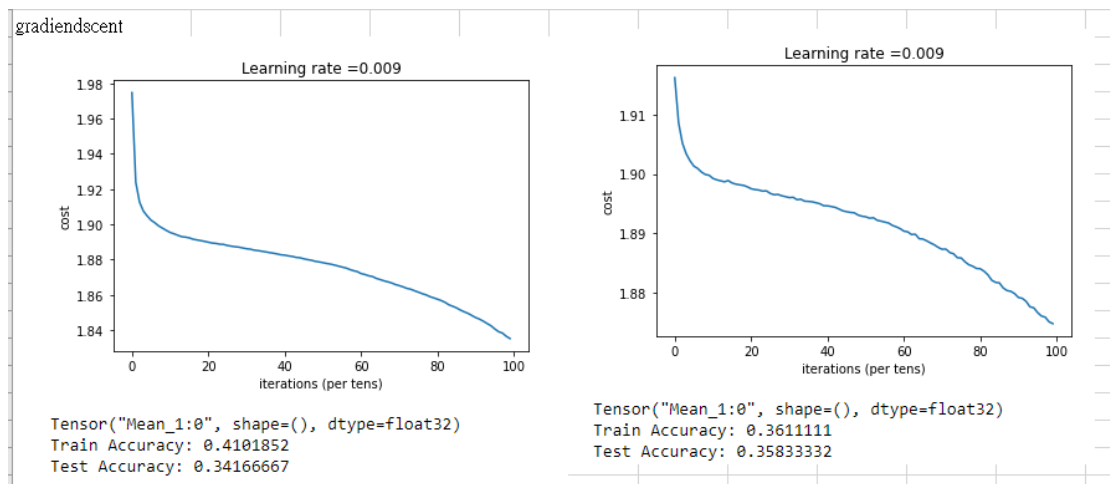
adam 能藉由增加 hidden layer 提高準確率，從 cost 圖可以判斷是否有必要增加訓練次數。



momentum 增加 Hidden layer 不一定能提高準確率，從 cost 圖可以判斷是否有必要增加訓練次數。



adagrad 訓練效率太差，cost 下降緩慢，需要調整 learning rate 或初始點才有不錯的效果，一味地增加 hidden layer 或學習次數效果並不明顯。

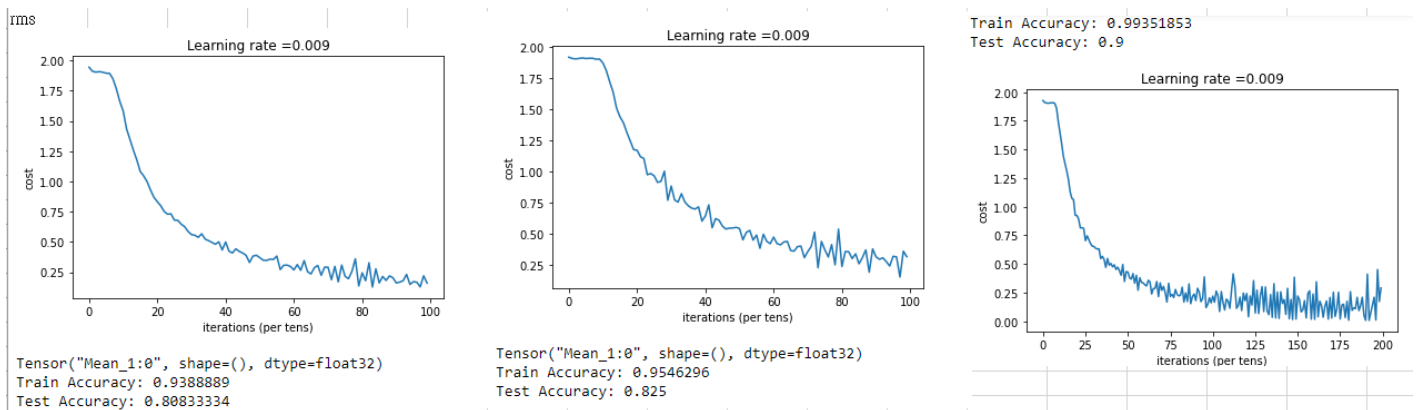


gradientdescent 的效果太差，需調整初始點和 learning rate 才有希望。

原始

加一層網路

epoch=200



RMSProp 的表現在這次資料很出色，非常適合，不用再額外修改 learning rate 和初始點，正確率都能在 8 成以上甚至 9 成。